

SPECIFICATION FOR THE DWARF WRITING LIBRARY

Draft #6

**Originally written by WATCOM International Corp.
Revised by Open Watcom contributors**

2026

Table of Contents

1 Debugging Information	1
1.1 Data Types	1
1.2 Initialization and Finalization	2
1.2.1 DWInit	2
1.2.2 DWFinis	3
1.2.3 DWBeginCompileUnit	3
1.2.4 DWEndCompileUnit	4
2 Ordering Considerations	5
3 Macro Information	7
3.1.1 DWMacStartFile	7
3.1.2 DWMacEndFile	7
3.1.3 DWMacDef	7
3.1.4 DWMacParam	7
3.1.5 DWMacFinis	7
3.1.6 DWMacUnDef	7
3.1.7 DWMacUse	8
4 File and Line-Number Management	9
4.1.1 DWSetFile	9
4.1.2 DWLineNum	9
4.1.3 DWDeclFile	9
4.1.4 DWDeclPos	10
4.1.5 DWReference	10
5 Location Expression Routines	11
5.1.1 DWLocInit	11
5.1.2 DWLocNewLabel	11
5.1.3 DWLocSetLabel	11
5.1.4 DWLocReg	11
5.1.5 DWLocStatic	12
5.1.6 DWLocSegment	12
5.1.7 DWLocConstU	12
5.1.8 DWLocConstS	12
5.1.9 DWLocOp0	12
5.1.10 DWLocOp	14
5.1.11 DWLocFinis	14
5.1.12 DWListInit	15
5.1.13 DWListEntry	15
5.1.14 DWListFinis	15
5.1.15 DWLocTrash	15
6 Typing Information	17
6.1.1 DWFundamental	17
6.1.2 DWModifier	18
6.1.3 DWTypedef	18
6.1.4 DWPointer	18
6.1.5 DWString	19
6.1.6 DWMemberPointer	19
6.2 Array Types	20
6.2.1 DWBeginArray	20

Table of Contents

6.2.2 DWArrayDimension	20
6.2.3 DWEndArray	21
6.3 Structure Types	21
6.3.1 DWStruct	21
6.3.2 DWBeginStruct	21
6.3.3 DWAddFriend	21
6.3.4 DWAddInheritance	22
6.3.5 DWAddField	22
6.3.6 DWAddBitField	22
6.3.7 DWEndStruct	23
6.4 Enumeration Types	23
6.4.1 DWBeginEnumeration	23
6.4.2 DWAddEnumerationConstant	23
6.4.3 DWEndEnumeration	23
6.5 Subroutine Type Declarations	24
6.5.1 DWBeginSubroutineType	24
6.5.2 DWEndSubroutineType	24
6.6 Lexical Blocks	24
6.6.1 DWBeginLexicalBlock	24
6.6.2 DWEndLexicalBlock	25
6.7 Common Blocks	25
6.7.1 DWBeginCommonBlock	25
6.7.2 DWEndCommonBlock	25
6.7.3 DWIncludeCommonBlock	25
6.8 Subroutines	25
6.8.1 DWBeginInlineSubroutine	25
6.8.2 DWBeginSubroutine	26
6.8.3 DWEndSubroutine	27
6.8.4 DWFormalParameter	27
6.8.5 DWEllipsis	28
6.8.6 DWLabel	28
6.8.7 DWVariable	28
6.8.8 DWConstant	28
6.8.9 DWAddress	29
6.8.10 DWPubname	29
7 Required Client Routines	31
7.1 Performance Considerations	31
7.1.1 CLISseek	32
7.1.2 CLITell	32
7.1.3 CLIReloc	32
7.1.4 CLIWrite	33
7.1.5 CLIAalloc	33
7.1.6 CLIFree	33
8 Examples	35
9 Revision History	39

1 Debugging Information

The include file "dw.h" should be included to access the DW library.

1.1 Data Types

The following types are defined in "dwcnf.h" and may be redefined if the entire library is to be recompiled ("dw.h" automatically includes "dwcnf.h").

<i>Type</i>	<i>Description</i>
<i>dw_sym_handle</i>	Has a client defined meaning; the DW library will pass these back to the client in <code>CLIRelocs</code> for <code>DW_W_STATIC</code> , and <code>DW_W_SEGMENT</code> .
<i>dw_targ_addr</i>	The contents of <code>dw_targ_addr</code> is unimportant to the DW library; it is only used for <code>sizeof(dw_targ_addr)</code> . A <code>dw_targ_addr</code> is the type that will be emitted for relocations to run-time addresses.
<i>dw_targ_seg</i>	This is the size of the quantity that <code>DW_W_SEGMENT</code> emits.
<i>dw_addr_offset</i>	The type used for offsets from some base address. For example, the <code>start_scope</code> parameter to typing routines, or the <code>addr</code> parameter to line number information. The code assumes this is an unsigned integer type.
<i>dw_addr_delta</i>	An integer type that can hold the largest possible difference between the <code>addr</code> parameter for two subsequent calls to <code>DWLineNum</code> .
<i>dw_linenum</i>	A line number. It must be an unsigned integer type.
<i>dw_linenum_delta</i>	<code>dw_linenum_delta</code> is a type that can hold the largest possible difference between two adjacent line numbers passed to <code>DWLineNum</code> or <code>DWReference</code> .
<i>dw_column</i>	A column number. It must be an unsigned integer type.
<i>dw_column_delta</i>	Type that can hold the largest possible difference between two adjacent column numbers passed to <code>DWReference</code> .
<i>dw_size_t</i>	Used for sizes of various things such as block constants (i.e. for <code>DWAddEnumerationConstant</code>) and the <code>size</code> parameter to <code>CLIWrite</code> .
<i>dw_uconst</i>	An unsigned integer type that can hold the largest possible unsigned integer constant.
<i>dw_sconst</i>	A signed integer type that can hold the largest possible signed integer constant.
<i>dw_sectnum</i>	Enumerated type that can hold all defined Dwarf sections, passed to client functions

dw_out_offset A integer type that can hold the largest possible section offset passed to `CLISseek` and returned by `CLITell`

dw_reloc_type A integer type that can hold the all relocation type

1.2 Initialization and Finalization

In the following functions, unless specified otherwise all strings are assumed to be null-terminated.

The DW library does not assume that a pointer passed to it is valid beyond the function call used to pass it. For example, you can pass the address of an auto-buffer that contains a string.

All names passed to the DW library should be unmangled.

The `cli` parameter required for all DW functions except `DWInit` is assumed to be a valid value returned by a call to `DWInit`.

Currently `DWENTRY` is defined to be nothing. It was created in case there is ever a need to put the DW library into a DLL.

1.2.1 DWInit

`dw_client DWENTRY DWInit( dw_init_info *info );`

Initialization for a compilation unit. Return an unique client id. This function will call client functions passed to it, so any client function initialization must be done before the call to `DWInit`.

```
typedef struct {
    void      (*reloc)( dw_sectnum, uint, ... );
    void      (*write)( dw_sectnum, const void *, dw_size_t );
    void      (*seek)( dw_sectnum, dw_out_offset, int );
    dw_out_offset (*tell)( dw_sectnum );
    void      (*alloc)( size_t );
    void      (*free)( void * );
} dw_funcs;

typedef struct {
    dw_lang      language;
    uint_8      compiler_options;
    const char   *producer_name;
    jmp_buf      exception_handler;
    dw_funcs     funcs;
} dw_init_info;
```

<i>Member</i>	<i>Description</i>
<i>language</i>	Language used.

<i>Constant</i>	<i>Language</i>
<i>DWLANG_C</i>	ISO/ANSI C
<i>DWLANG_CPP</i>	C++
<i>DWLANG_FORTRAN</i>	FORTRAN77

compile_options Compilation option, which is a combination of bits:

<i>Bit</i>	<i>Description</i>
<i>DW_CM_BROWSER</i>	The library generates the debugging information for the class browser.
<i>DW_CM_DEBUGGER</i>	The library generates the debugging information for the debugger.
<i>DW_CM_UPPER</i>	For FORTRAN - The compiler converts all identifier names to upper case.
<i>DW_CM_LOWER</i>	For FORTRAN - The compiler converts all identifier names to lower case.

producer A string that identifies the compiler.

exception_handler If the library ends up in a situation which it can't handle (can we say bug ;-)) this jmp_buf will be called with a non-zero value. This is a fatal exit, and the client should not call any of the DW functions.

FIXME: The library is currently not very good at cleaning up memory in these situations.

funcs These functions are described in a later section. The initialization routines may call any of them; so any initialization necessary for these routines must be done before DWInit is called.

The details of the above functions are discussed in Part 3.

1.2.2 DWFin

```
void DWENTRY DWFin( dw_client cli );
```

Finalize the debugging information generator. This routine must be called last. It frees any structures that the DW library required, and flushes all the debugging information.

1.2.3 DWBeginCompileUnit

```
dw_handle DWENTRY DWBeginCompileUnit( dw_client cli, const char *source_filename, const char *directory, dw_loc_handle segment, const unsigned offset_size );
```

This function is called some time after DWInit. The only other DW functions that can be called in between are those dealing with location expressions.

Parameter

source_filename Name of the source file.

directory Compilation directory.

segment A location expression whose result is the code segment portion of the low_pc and high_pc.

offset_size The size in bytes of the offset portion of an address in this compile unit

The following CLIRelocs will be required:

DW_W_HIGH_PC

DW_W_LOW_PC

DW_W_SECTION_POS

DW_W_UNIT_SIZE

1.2.4 DWEndCompileUnit

```
void DWENTRY DWEndCompileUnit( dw_client cli );
```

This function pairs up with DWBeginCompileUnit. After this, until the next DWBeginCompileUnit, the only valid calls are those made to location expression routines (or DWFin).

2 Ordering Considerations

In general the DW routines are called in an order that matches the order of the declarations during the source program. The sole exception to this are the Macro information routines. Since it is possible to have a separate preprocessor pass, the library assumes that these routines can be called before any of the other routines. That is why the macro routines have a separate mechanism for specifying file and line number.

3 Macro Information

3.1.1 *DWMacStartFile*

void DWENTRY DWMacStartFile(dw_client cli, dw_linenum line, const char *name);

Subsequent DWMac calls refer to the named file.

3.1.2 *DWMacEndFile*

void DWENTRY DWMacEndFile(dw_client cli);

End the current included file.

3.1.3 *DWMacDef*

dw_macro DWENTRY DWMacDef(dw_client cli, dw_linenum line, const char *name);

Defines a macro. `name` is the name of the macro. A `dw_macro` is returned and must be used in a subsequent call to `DWMacFini` (and possibly `DWMacParam`).

3.1.4 *DWMacParam*

void DWENTRY DWMacParam(dw_client cli, dw_macro mac, const char *name);

Adds a parameter to the macro definition `mac`. `name` is the name of the parameter with no leading or trailing white-space. The order of parameters must be the same as they appear in the source program.

3.1.5 *DWMacFini*

void DWENTRY DWMacFini(dw_client cli, dw_macro mac, const char *def);

Finishes the macro definition `mac`. `def` is the definition string.

3.1.6 *DWMacUnDef*

void DWENTRY DWMacUnDef(dw_client cli, dw_linenum line, const char *name);

Undefines the macro named `name`.

3.1.7 DWMacUse

```
void DWENTRY DWMacUse( dw_client cli, dw_linenum line, const char *name );
```

Indicate where the macro named `name` is used.

4 File and Line-Number Management

4.1.1 DWSetFile

```
void DWENTRY DWSetFile( dw_client cli, const char *file );
```

Specifies the current file. The default is for the `source_filename` parameter from the `dw_init_info` to be the current file.

4.1.2 DWLineNum

```
void DWLineNum( dw_client cli, uint info, dw_linenum line, uint col, dw_addr_offset addr );
```

Sets the current source line number and machine address. The line numbers information of all instructions, not just declarations, are stored by this routine. Note that all source line numbers are relative to the beginning of their corresponding source file. So the line number of the first line of an included file is one.

Parameter

info The information about the line, which is established by the combination of the following bits:

<i>Bit</i>	<i>Description</i>
<i>DW_LN_DEFAULT</i>	There is no special information about the line.
<i>DW_LN_STMT</i>	The line is the beginning of a statement.
<i>DW_LN_BLK</i>	The line is the beginning of a block.

line Source line number, numbered beginning with one on the first line of the file.

col Source column number, which begins at 1.

addr Address of instruction relative to the beginning of the compilation unit. If it is at all possible the client should call `DWLineNum` with increasing `addrs`. The line parameter does not have to be increasing. The size of the emitted line number information is smaller if increasing `addrs` are used (There is also an implementation limitation that the maximum decrease of `addr` between two calls is 32768).

4.1.3 DWDeclFile

```
void DWENTRY DWDeclFile( dw_client cli, const char *name );
```

Subsequent declarations are from the file named `name`.

4.1.4 DWDeclPos

```
void DWENTRY DWDeclPos( dw_client cli, dw_linenum line, dw_column column );
```

The next declaration occurs at the indicated line and column in the source file set by the last call to `DWDeclFile`. Note that the position is only used for the immediate next declaration. If there are multiple declarations on the same line, then multiple calls should be made.

4.1.5 DWReference

```
void DWENTRY DWReference( dw_client cli, dw_linenum line, dw_column column, dw_handle  
dependant );
```

Indicate that in the source code there is a reference to the dependant. This reference is attributed to the current scope of debugging information. (i.e., if it is done inside a structure, then the structure is considered to be the "referencer").

5 Location Expression Routines

Many functions require a `dw_loc_handle`. These are handles for expressions that the debugger will evaluate. A `dw_loc_handle` can be either a single expression, or a list of expressions created by `DWListFini`. The BROWSER is only interested in whether a location expression is present or not; so when creating BROWSER output the client may create an empty location expression and use that wherever appropriate.

The expressions are evaluated on a stack machine, with operations described later. In some cases the stack will be initially empty, in other cases (such as when calculating the address of a structure field) some base address will be on the stack.

A location expression is limited to roughly 64K. Since each op-code is a single byte, this shouldn't pose much of a limitation (famous last words). The destination of the branch instructions `DW_LOC_BRA` and `DW_LOC_SKIP` must be within 32K of the current instruction (This is a limitation of the DWARF format, not a limitation of the DW library).

5.1.1 DWLocInit

```
dw_loc_id DWENTRY DWLocInit( dw_client cli );
```

First function called to create a location expression for a symbol. An unique `dw_loc_id` is returned to the front end.

5.1.2 DWLocNewLabel

```
dw_loc_label DWENTRY DWLocNewLabel( dw_client cli, dw_loc_id loc );
```

Create a label for the location expression being built in `loc`. This label can be used for forward or backward references by `DW_LOC_SKIP` and `DW_LOC_BRA`.

5.1.3 DWLocSetLabel

```
void DWENTRY DWLocSetLabel( dw_client cli, dw_loc_id loc, dw_loc_label label );
```

Give the label `label` the address of the next operation emitted into the location expression `loc`.

5.1.4 DWLocReg

```
void DWENTRY DWLocReg( dw_client cli, dw_loc_id loc, uint reg );
```

This 'operation' informs the debugger that the value it seeks is in the register named by `reg`.

FIXME: need to define the possible values of `reg`.

5.1.5 DWLocStatic

```
void DWENTRY DWLocStatic( dw_client cli, dw_loc_id loc, dw_sym_handle sym );
```

This operation pushes the address of `sym` on the stack.

5.1.6 DWLocSegment

```
void DWENTRY DWLocSegment( dw_client cli, dw_loc_id loc, dw_sym_handle sym );
```

This operation pushes the segment of the address of `sym` on the stack.

5.1.7 DWLocConstU

```
void DWENTRY DWLocConstU( dw_client cli, dw_loc_id loc, dw_uconst value );
```

Pushes an atom which is has an unsigned constant value `value`.

5.1.8 DWLocConstS

```
void DWENTRY DWLocConstS( dw_client cli, dw_loc_id loc, dw_sconst value );
```

Pushes an atom which is has a signed constant value `value`.

5.1.9 DWLocOp0

```
void DWENTRY DWLocOp0( dw_client cli, dw_loc_id loc, dw_loc_op op );
```

Performs one of the operations listed below.

<i>Operation</i>	<i>Description</i>
<i>DW_LOC_ABS</i>	It pops the top stack entry and pushes its absolute value.
<i>DW_LOC_AND</i>	It pops the top two stack values, performs the logical AND operation on the two, and pushes the result.
<i>DW_LOC_DEREF</i>	It pops the top stack entry and treats it as an address. The value retrieved from that address is pushed. The size of data retrieved from the dereferenced address is an addressing unit.
<i>DW_LOC_DIV</i>	It pops the top two stack values, divides the former second entry by the former top of the stack using signed division, and pushes the result.
<i>DW_LOC_DROP</i>	It pops the value at the top of the stack.
<i>DW_LOC_DUP</i>	It duplicates the value at the top of the stack.

<i>DW_LOC_EQ</i>	Pop two entries from stack, push 1 if they are equal; push 0 otherwise.
<i>DW_LOC_GE, DW_LOC_GT, DW_LOC_LE, DW_LOC_LT</i>	These operation pop the top two stack values, compare the former top of stack from the former second entry, and pushes 1 onto stack if the comparison is true, 0 if it is false. The comparisons are signed comparison.
<i>DW_LOC_MINUS</i>	It pops the top two stack values, subtracts the former top of the stack from the former second entry, and pushes the result.
<i>DW_LOC_MOD</i>	It pops the top two stack values, and pushes the result of the calculation: former second stack entry modulo the former top of the stack.
<i>DW_LOC_MUL</i>	It pops the top two stack values, multiplies them together, and pushes the result.
<i>DW_LOC_NE</i>	Pop two entries from stack, push 0 if they are equal; push 1 otherwise.
<i>DW_LOC_NEG</i>	It pops the top value and pushes its negation.
<i>DW_LOC_NOP</i>	A placeholder; has no side-effects.
<i>DW_LOC_NOT</i>	It pops the top value and pushes its logical complement.
<i>DW_LOC_OR</i>	It pops the top two stack entries, performs the logical OR operation on them, and pushes the result.
<i>DW_LOC_OVER</i>	It duplicates the entry currently second in the stack at the top of the stack.
<i>DW_LOC_PLUS</i>	It pops the top two stack entries, and pushes their sum.
<i>DW_LOC_ROT</i>	It rotates the first three stack entries. The entry at the top of the stack becomes the third entry, the second entry becomes the top, and the third entry becomes the second.
<i>DW_LOC_SHL</i>	It pops the top two stack entries, shifts the former second entry left by the number of bits specified by the former top of the stack, and pushes the result.
<i>DW_LOC_SHR</i>	It pops the top two stack entries, shifts the former second entry right (logically) by the number of bits specified by the former top of the stack, and pushes the result.
<i>DW_LOC_SHRA</i>	It pops the top two stack entries, shifts the former second entry right (arithmetically) by the number of bits specified by the former top of the stack, and pushes the result.
<i>DW_LOC_SWAP</i>	It swaps the top two stack entries.

DW_LOC_XDEREF

It provides an extended dereference mechanism. The entry at the top of the stack is treated as an address. The second stack entry is treated as an "address space identifier" for those architectures that support multiple address spaces. The top two stack elements are popped, a data item is retrieved through an implementation-defined address calculation and pushed as the new stack top. The size of data retrieved is an addressing unit.

DW_LOC_XOR

It pops the top two stack entries, performs the logical EXCLUSIVE-OR operation on them, and pushes the result.

5.1.10 DWLocOp

```
void DWENTRY DWLocOp( dw_client cli, dw_loc_id loc, dw_loc_op op, ... );
```

Performs one of the following operations:

Operation

Description

DW_LOC_BRA

It is followed by a *dw_loc_label* operand. This operation pops the top stack entry, if the value is not zero, then jump to the label.

DW_LOC_BREG

Followed by two operands, the first is a register, and the second is an *dw_sconst* to add to the value in the register. The result is pushed onto the stack.

DW_LOC_FBREG

Takes one *dw_sconst* parameter which is added to the value calculated by the *frame_base_loc* parameter to the current subroutine, then pushed on the stack.

DW_LOC_PICK

It is followed by a *uint* operand which is an index. The stack entry with the specified index (0 through 255, inclusive; 0 means the top) is pushed on the stack.

DW_LOC_PLUS_UCONST

It is followed an *dw_uconst* operand. It pops the top stack entry, adds it to the operand and pushes the result.

DW_LOC_SKIP

It is followed by a *dw_loc_label* operand. Control is transferred immediately to this label.

5.1.11 DWLocFini

```
dw_loc_handle DWENTRY DWLocFini( dw_client cli, dw_loc_id loc );
```

Ends the location expression for a symbol, and returns a handle that may be passed to other DW routines.

5.1.12 DWListInit

```
dw_list_id DWENTRY DWListInit( dw_client cli );
```

First function called to create a location list for a symbol.

5.1.13 DWListEntry

```
void DWENTRY DWListEntry( dw_client cli, dw_list_id id, dw_sym_handle beg, dw_sym_handle  
end, dw_loc_handle loc );
```

Define an entry in the location list.

Parameter

<i>beg</i>	A beginning address. This address is relative to the base address of the compilation unit referencing this location list. It marks the beginning of the range over which the location is valid.
<i>end</i>	A ending address. This address is relative to the base address of the compilation unit referencing this location list. It marks the first address past the end of the range over which the location is valid. Overlapping ranges are possible and are interpreted to mean that the value may be found in one of many places during the overlap. A CLIReloc for DW_W_LABEL will be made for each dw_sym_handle.
<i>loc</i>	A location expression describing the location of the object over the range specified by the beginning and end addresses.

5.1.14 DWListFini

```
dw_loc_handle DWENTRY DWListFini( dw_client cli, dw_list_id );
```

Finishes the creation of the location list.

5.1.15 DWLocTrash

```
void DWENTRY DWLocTrash( dw_client cli, dw_loc_handle loc );
```

Frees the memory associated with the location expression or list loc. A location expression/list can be created and used over and over again until it is freed by calling this function.

6 Typing Information

Unless otherwise noted, calls to these functions emit debugging information immediately. The DWARF format requires that debugging information appear in the same order as it does in the source code. So, for example, a structure's fields must be created in the same order that they appear in the source program.

Some of the following functions have common parameters. Here is the documentation for these common parameters:

Parameter

char *name A null-terminated type name. i.e., "struct foobar {}" has the name foobar. If this parm is NULL then no name is emitted.

dw_addr_offset start_scope This is the offset from the low_pc value for the enclosing block that the declaration occurs at. This is most commonly 0.

dw_flags flags Some routines have additional flags available here; but unless otherwise noted, the following are always available:

<i>Flag</i>	<i>Description</i>
-------------	--------------------

<i>DW_FLAG_DECLARATION</i>	The object is a declaration, not a definition
-----------------------------------	---

<i>DW_FLAG_PRIVATE</i>	The object has the C++ private attribute.
-------------------------------	---

<i>DW_FLAG_PROTECTED</i>	The object has the C++ protected attribute.
---------------------------------	---

<i>DW_FLAG_PUBLIC</i>	The object has the C++ public attribute.
------------------------------	--

6.1.1 DWFundamental

```
dw_handle DWENTRY DWFundamental( dw_client cli, char * name, dw_ftype fund_idx, unsigned size );
```

Get a handle for a fundamental type. fund_idx is one of the following:

DW_FT_ADDRESS

DW_FT_BOOLEAN

DW_FT_COMPLEX_FLOAT

DW_FT_FLOAT

DW_FT_SIGNED

DW_FT_SIGNED_CHAR

DW_FT_UNSIGNED

DW_FT_UNSIGNED_CHAR

For convenience, DW_FT_MIN, and DW_FT_MAX are defined. A valid fundamental type is in the range DW_FT_MIN <= ft < DW_FT_MAX. The DW library will not return the same handle when called with the same base type (so the client must store the base type handlers).

Parameter

name The name of the type being defined.

size The size in bytes of the type being defined.

6.1.2 DWModifier

dw_handle DWENTRY DWModifier(dw_client cli, dw_handle base_type, uint modifiers);

Specifies a modifier to a type. *base_type* is the base type to be modified with the modifier *modifier*. The available modifiers are:

<i>Modifier Constant</i>	<i>Description</i>
<i>DW_MOD_CONSTANT</i>	The object is a constant
<i>DW_MOD_VOLATILE</i>	The object is volatile.
<i>DW_MOD_NEAR</i>	The object is a near object.
<i>DW_MOD_FAR</i>	The object is a far object.
<i>DW_MOD_HUGE</i>	The object is a huge object.
<i>DW_MOD_FAR16</i>	The object is a far16 object.

6.1.3 DWTypedef

dw_handle DWENTRY DWTypedef(dw_client cli, dw_handle base_type, const char *name, dw_addr_offset start_scope, dw_flags flags);

This function gives a name to a type. The *name* must not be NULL. The flag value DW_FLAG_DECLARATION is not allowed.

6.1.4 DWPointer

dw_handle DWENTRY DWPointer(dw_client cli, dw_handle base_type, dw_flags flags);

Declares a pointer type.

Parameter

base_type The pointed-at type.

flags Only the following flags are available:

<i>Flags</i>	<i>Description</i>
--------------	--------------------

<i>DW_FLAG_REFERENCE</i>	Declare a pointer that is dereferenced automatically.
--------------------------	---

<i>DW_FLAG_PTR_TYPE_NORMAL</i>	A normal pointer (i.e. a model dependant pointer).
--------------------------------	--

<i>DW_FLAG_PTR_TYPE_NEAR16</i>	A near 16-bit pointer.
--------------------------------	------------------------

<i>DW_FLAG_PTR_TYPE_FAR16</i>	A far 16-bit pointer.
-------------------------------	-----------------------

<i>DW_FLAG_PTR_TYPE_HUGE</i>	A huge 16-bit pointer.
------------------------------	------------------------

<i>DW_FLAG_PTR_TYPE_NEAR32</i>	A near 32-bit pointer.
--------------------------------	------------------------

<i>DW_FLAG_PTR_TYPE_FAR32</i>	A far 32-bit pointer.
-------------------------------	-----------------------

6.1.5 DWString

dw_handle DWENTRY DWString(dw_client cli, dw_loc_handle string_length, dw_size_t byte_size, const char *name, dw_addr_offset start_scope, dw_flags flags);

Declares a type to be a block of characters.

Parameter

string_length If this parameter is non-NULL then it is a location expression that the debugger executes to get the address where the length of the string is stored in the program. In this case the *byte_size* parameter describes the number of bytes to be retrieved at the location calculated. If *byte_size* is 0, then the debugger will use `sizeof( long )`.

byte_size If *string_length* is NULL then this parameter is the number of bytes in the string. Otherwise see *string_length*.

6.1.6 DWMemberPointer

dw_handle DWENTRY DWMemberPointer(dw_client cli, dw_handle containing_struct, dw_loc_handle use_location, dw_handle base_type, const char *name, dw_flags flags);

Declares a C++ pointer type to a data or function member of a class or structure.

Parameter

containing_struct A handle to the class or struct to whose members objects of this type may point.

use_location This refers to the location expression which describes how to get to the member it points to from the beginning of the entire class. It expects the base address of the structure/class object to be on the stack before the debugger starts to execute the location description.

base_type The type of the member to which this object may point to.

6.2 Array Types

6.2.1 DWBeginArray

```
dw_handle DWENTRY DWBeginArray( dw_client cli, dw_handle elt_type, uint stride_size, const
char *name, dw_addr_offset scope, dw_flags flags );
```

Begin the declaration of an array. This function call must be followed by calls to `DWArrayDimension` and `DWEndArray`.

Parameter

elt_type Handle for the type of the elements of this array.

stride_size If this value is non-zero then it indicates the number of bits of each element of the array (Useful if the number of bits used to store an element in the array is different from the number of bits used to store an individual element of type `elt_type`).

6.2.2 DWArrayDimension

```
void DWENTRY DWArrayDimension( dw_client cli, const dw_dim_info *info );
```

Add a dimension to the previously started array. This function must be called for each dimension in the order that the dimensions appear in the source program. `info` points to an instance of the following structure:

```
typedef struct {
    dw_handle    index_type;
    dw_uconst    lo_data;
    dw_uconst    hi_data;
} dw_dim_info;
```

<i>Field</i>	<i>Description</i>
---------------------	---------------------------

<i>hi_bound_fmt</i>	This is similar to <code>lo_bound_fmt</code> but describes the high bound of this dimension.
----------------------------	--

<i>index_type</i>	This is the handle of the type of the indicies for this dimension.
--------------------------	--

<i>lo_data</i>	The low bound of the array.
-----------------------	-----------------------------

<i>hi_data</i>	The upper bound of the array.
-----------------------	-------------------------------

6.2.3 DWEndArray

```
void DWENTRY DWEndArray( dw_client cli, dw_handle array_hdl, dw_handle elt_type, uint
stride_size, const char *name, dw_addr_offset scope, dw_flags flags );
```

This finishes the writing of the record to describe the array. A sufficient number of calls to `DWArrayDimension` must have been made before `DWEndArray` is called.

6.3 Structure Types

6.3.1 DWStruct

```
dw_handle DWENTRY DWStruct( dw_client cli, uint kind );
```

Create a handle for a structure type that will be defined later. This handle can be used for other DW routines even before `DWBeginStruct` has been called.

<i>Kind</i>	<i>Description</i>
-------------	--------------------

<i>DW_ST_CLASS</i>	A C++ class type.
--------------------	-------------------

<i>DW_ST_STRUCT</i>	A structure type.
---------------------	-------------------

<i>DW_ST_UNION</i>	A union type.
--------------------	---------------

6.3.2 DWBeginStruct

```
void DWENTRY DWBeginStruct( dw_client cli, dw_handle struct_hdl, dw_size_t size, const char
*name, dw_addr_offset scope, dw_flags flags );
```

Begin the declaration of the structure reserved by a call to `DWStruct`. This function begins a nesting of the debugging information. Subsequent calls, up to the corresponding `DWEndStruct` call, to the DW library become children of this structure. i.e., this function marks the beginning of the scope of the structure definition.

Parameter

<i>struct_hdl</i>	A <code>dw_handle</code> returned by a call to <code>DWStruct</code> .
-------------------	--

<i>size</i>	If this is non-zero it indicates the number of bytes required to hold an element of this structure including any padding bytes.
-------------	---

6.3.3 DWAddFriend

```
void DWENTRY DWAddFriend( dw_client cli, dw_handle friend );
```

Add `friend` as a friend to the current structure.

6.3.4 DWAddInheritance

```
dw_handle DWENTRY DWAddInheritance( dw_client cli, dw_handle ancestor, dw_loc_handle loc,  
dw_flags flags );
```

Indicate the the current structure inherits from another structure.

Parameter

<i>ancestor</i>	The handle of the ancestor to be inherited.
<i>loc</i>	A location expression that describes the location of the beginning of the data members contributed to the entire class by the ancestor relative to the beginning of the address of the data members of the entire class.
<i>flags</i>	In addition to the common values of <code>flags</code> , the flag <code>DW_FLAG_VIRTUAL</code> may be supplied to indicate that the inheritance serves as a virtual base class. As well, the flag <code>DW_FLAG_DECLARATION</code> is not allowed here.

6.3.5 DWAddField

```
dw_handle DWENTRY DWAddField( dw_client cli, dw_handle field_hdl, dw_loc_handle loc, const  
char *name, dw_flags flags );
```

Add a data member to a structure.

Parameter

<i>field_hdl</i>	The <code>dw_handle</code> of the type of this field.
<i>loc</i>	A location expression which expects the base address of the structure to be pushed on the stack and calculates the base address of this field. If the structure is a union type, then this parameter may be NULL. If this is a static data member of a class then this parameter may be NULL if the actual definition of the parameter is outside the class.
<i>flags</i>	The additional flag <code>DW_FLAG_STATIC</code> may be used to indicate a static structure member.

6.3.6 DWAddBitField

```
dw_handle DWENTRY DWAddBitField( dw_client cli, dw_handle field_hdl, dw_loc_handle loc,  
dw_size_t byte_size, uint bit_offset, uint bit_size, const char *name, dw_flags flags );
```

Add a bitfield member to a structure.

Parameter

<i>field_hdl</i>	the <code>dw_handle</code> of the type of this field.
<i>loc</i>	A location expression which expects the base address of the structure most closely containing the bit field to be pushed and the stack, and which calculates the base address of this field.

<i>byte_size</i>	This field must be the non-zero byte size of the unit of storage containing the bit-field. This is required only if the storage required cannot be determined by the type of the bit-field (i.e., padding bytes). If the size can be determined by the type of the bit-field, then this value may be 0.
<i>bit_offset</i>	The number of bits to the left of the leftmost (most significant); bit of the bit field value.
<i>bit_size</i>	The number of bits occupied by this bit-field value.

6.3.7 DWEndStruct

```
void DWENTRY DWEndStruct( dw_client cli );
```

End the current structure. Client must ensure proper Begin/End matching.

6.4 Enumeration Types

6.4.1 DWBeginEnumeration

```
dw_handle DWENTRY DWBeginEnumeration( dw_client cli, dw_size_t byte_size, const char *name,
dw_addr_offset scope, dw_flags flags );
```

Begin the definition of an enumerated type. *byte_size* is the number of bytes required to hold an instance of this enumeration. This call must be followed by calls to *DWAddEnumerationConstant* and *DWEndEnumeration*. No other DW calls may be made before the call to *DWEndEnumeration*. The DWARF standard requires that the constants be defined in *reverse* order to which they appear in the source program.

6.4.2 DWAddEnumerationConstant

```
void DWENTRY DWAddEnumerationConstant( dw_client cli, dw_uconst value, const char *name );
```

Add the constant value (that is *byte_size* bytes large as determined by the parameter to *DWBeginEnumeration*); with the name *name* to the current enumeration.

6.4.3 DWEndEnumeration

```
void DWENTRY DWEndEnumeration( dw_client cli );
```

Finish the current enumeration.

6.5 Subroutine Type Declarations

These function calls deal with declarations of subroutines. That is, their prototypes, or for use in creating function pointers.

6.5.1 DWBeginSubroutineType

```
dw_handle DWENTRY DWBeginSubroutineType( dw_client cli, dw_handle return_type, const char  
*name, dw_addr_offset scope, dw_flags flags );
```

Begin the nested declaration of the subroutine type. All calls to the DW library after this, until `DWEndSubroutineType` are in the scope of the declaration of the subroutine type (i.e., if it's a prototyped C function, then declarations before `DWEndSubroutineType` are similar to declarations inside the prototype). Parameters for this type are declared using the entries `DWAddParmToSubroutineType` and `DWAddEllipsisToSubroutineType`.

Parameter

return_type If the function is void, this parameter must be NULL. Otherwise it is a handle for the return type of the subroutine.

flags In addition to the standard flags, `DW_FLAG_PROTOTYPED` indicates that the declaration of the subroutine type was prototyped in the source code. As well, the "address class" set of flags used in `DWPointer` are also allowed here (e.g. `DW_TYPE_FAR16` etc.)

6.5.2 DWEndSubroutineType

```
void DWENTRY DWEndSubroutineType( dw_client cli );
```

The client must ensure that proper Begin/End matching is done.

6.6 Lexical Blocks

6.6.1 DWBeginLexicalBlock

```
dw_handle DWENTRY DWBeginLexicalBlock( dw_client cli, dw_loc_handle segment, const char  
*name );
```

Begin a new lexical scope. `name` may be NULL indicating an un-named scope. Two `CLIReloc` calls will be made, one for `DW_W_LOW_PC` and one for `DW_W_HIGH_PC` which indicate the first byte of the scope, and the first byte beyond the end of the scope. `segment` if non-null is an expression that evaluates to the segment this block is in.

6.6.2 DWEndLexicalBlock

```
void DWENTRY DWEndLexicalBlock( dw_client cli );
```

End a lexical scope. As usual, the client must ensure that Begin/End pairs match.

6.7 Common Blocks

6.7.1 DWBeginCommonBlock

```
dw_handle DWENTRY DWBeginCommonBlock( dw_client cli, dw_loc_handle loc, dw_loc_handle  
segment, const char *name, unsigned flag );
```

Begin the declarations for the common block named `name` and located at `loc`. `segment` if non-null indicates which segment the common block is in. The only flag that is valid for the `flag` parameter is `DW_FLAG_DECLARATION`.

6.7.2 DWEndCommonBlock

```
void DWENTRY DWEndCommonBlock( dw_client cli );
```

End of declarations for the common block.

6.7.3 DWIncludeCommonBlock

```
dw_handle DWENTRY DWIncludeCommonBlock( dw_client cli, dw_handle common_block );
```

Used in the subroutine scope that references the common block.

6.8 Subroutines

6.8.1 DWBeginInlineSubroutine

```
dw_handle DWENTRY DWBeginInlineSubroutine( dw_client cli, dw_handle out_of_line,  
dw_loc_handle ret_addr, dw_loc_handle segment );
```

Begin a definition of a particular instance of an inlined subroutine. `out_of_line` is a handle to the "out of line" instance of the subroutine (i.e., a handle from a `DWBeginSubroutine` call that had the `DW_FLAG_OUT_OF_LINE` flag). Each instance of the inlined subroutine must have its own copies of entries describing parameters to that subroutine and its local variables. `ret_addr` gives the location of the return address (if any). `segment` if non-null indicates which segment the expansion occurs in.

6.8.2 DWBeginSubroutine

```
dw_handle DWENTRY DWBeginSubroutine( dw_client cli, dw_call_type call_type, dw_handle
return_type, dw_loc_handle return_addr_loc, dw_loc_handle frame_base_loc, dw_loc_handle
structure_loc, dw_handle member_hdl, dw_loc_handle segment, const char *name, dw_addr_offset
start_scope, dw_flags flags );
```

Begin a declaration/definition of a subroutine or entry point. This begins a nesting of the debugging information, and must be followed by calls to `DWFormalParameter` et al to declare the parameters, types, and variables for this subroutine. Unless `DW_FLAG_DECLARATION` is set, this will require a `DW_W_LOW_PC` and/or a `DW_W_HIGH_PC`.

Parameter

call_type Not currently used, but should be one of:

DW_SB_NEAR_CALL

DW_SB_FAR_CALL

DW_SB_FAR16_CALL

return_type Handle for the return type. Must be NULL for void-type subroutines.

return_addr_loc If non-NULL then this is a location expression that calculates the address of memory that stores the return address.

frame_base_loc If non-NULL then this is a location expression that describes the "frame base" for the subroutine or entry point (If the frame base changes during the subroutine, it might be desirable for local variables to be calculated from the frame base, and then use a location list for the frame base).

structure_loc For member functions of structure types, this calculates the address of the slot for the function within the virtual function table for the enclosing class or structure.

member_hdl If this is a definition of a member function occurring outside the body of the structure type, then this is the handle for the type definition of the structure.

segment If non-null then this is a location expression that evaluates to the segment for this subprogram.

The following additional flags are available:

flag description

DW_FLAG_PROTOTYPED The function was declared with ANSI-C style prototyping, as opposed to K&R-C style parameter lists.

DW_FLAG_ARTIFICIAL The function was created by the compiler (i.e. not explicitly declared in any of the user's source files)

DW_FLAG_VIRTUAL This is a virtual subroutine.

DW_FLAG_PURE_VIRTUAL This is a pure virtual subroutine.

DW_FLAG_MAIN For Fortran PROGRAM-type subroutines.

DW_FLAG_SUB_STATIC A file static subroutine or function. Also used for a static member function, and for nested subroutine declarations.

DW_FLAG_WAS_INLINED The function was generated inline by the compiler.

DW_FLAG_DECLARED_INLINED The function was declared inline by the user.

6.8.3 DWEndSubroutine

```
void DWENTRY DWEndSubroutine( dw_client cli );
```

End the current nesting of DWBeginSubroutine or DWBeginInlineSubroutine.

6.8.4 DWFormalParameter

```
dw_handle DWENTRY DWFormalParameter( dw_client cli, dw_handle parm_type, dw_loc_handle
parm_loc, dw_loc_handle segment, const char *name, uint default_value_type, ... );
```

Declare a formal parameter to the current function.

parm_type The type of the parameter.

parm_loc A location description that yields the address of the parameter. May be NULL indicating unknown address.

segment A location expression that yields the segment of the parameter. May be NULL indicating the default segment.

default_value_type One of the following:

DW_DEFAULT_NONE There is no default value for this parameter.

DW_DEFAULT_FUNCTION The default value for this parameter is returned by a function with no args, that is specified by a CLIREloc for DW_W_DEFAULT_FUNCTION.

DW_DEFAULT_STRING The default value is a null-terminated string that is specified as an extra parameter to this DWFormalParameter.

DW_DEFAULT_BLOCK The default value is a constant block of data that is specified by extra "const void *" and "dw_size_t" parameters to DWFormalParameter.

... Extra parameters depend on the default_value_type.

6.8.5 DWEllipsis

```
dw_handle DWENTRY DWEllipsis( dw_client cli );
```

Indicate that the current subroutine has unspecified parameters. Used for "..." in C.

6.8.6 DWLabel

```
dw_handle DWENTRY DWLabel( dw_client cli, dw_loc_handle segment, const char *name,  
dw_addr_offset start_scope );
```

Declare a label inside a subroutine. `start_scope` will usually be 0, but is here for future compatibility. A CLIReloc for DW_W_LABEL will be made. `segment` if non-null indicates which segment the label belongs to.

6.8.7 DWVariable

```
dw_handle DWENTRY DWVariable( dw_client cli, dw_handle type, dw_loc_handle loc, dw_handle  
member_of, dw_loc_handle segment, const char *name, dw_addr_offset start_scope, dw_flags flags );
```

Declare a variable.

<i>type</i>	The type of this variable.
<i>loc</i>	A location expression yielding the address of this variable.
<i>member_of</i>	If this is the definition of a static data member then this is the handle to the structure type. Otherwise this is NULL.
<i>segment</i>	If this is non-null then it evaluates to the segment the variable is in.
<i>flags</i>	If DW_FLAG_GLOBAL is set then this is a global variable. Otherwise it is a local variable. File static variables in C and C++ are considered local variables. If DW_FLAG_ARTIFICIAL is set then this is a variable that has been created by the compiler.

6.8.8 DWConstant

```
dw_handle DWENTRY DWConstant( dw_client cli, dw_handle type, const void *value, dw_size_t  
len, dw_handle member_of, const char *name, dw_addr_offset start_scope, dw_flags flags );
```

Declare a named constant.

<i>type</i>	The type of this constant.
<i>value</i>	Pointer to the value for this constant.
<i>len</i>	The length of this constant. If <code>len</code> is 0, then <code>value</code> is considered to be a null-terminated string.

member_of If this is the definition of a constant member of a structure type, then this is the handle to the structure type. Otherwise it is NULL.

6.8.9 DWAddress

```
void DWENTRY DWAddress( dw_client cli, uint_32 len );
```

DWARF builds a table of all the addresses attributed to a compilation unit. The client calls this function to add addresses to this table. `len` is the length of this address range. The base of the address range is filled in by a CLIReloc for `DW_W_ARANGE_ADDR`.

6.8.10 DWPubname

```
void DWENTRY DWPubname( dw_client cli, dw_handle hdl, const char *name );
```

These are used to speed up the debugger. This should be called for any name that has global scope. `hdl` is the handle for the debugging entry that declares/defines the `name`.

7 Required Client Routines

The debugging information has several sections indicated by the following enumerated type:

<i>Constant</i>	<i>Description</i>
<i>DW_DEBUG_INFO</i>	This section is called <i>.debug_info</i> , which stores all the debugging information entries.
<i>DW_DEBUG_PUBNAMES</i>	This section is called <i>.debug_pubnames</i> , which stores a table consisting of object name information that is used in lookup by Name.
<i>DW_DEBUG_ARANGES</i>	This section is called <i>.debug_aranges</i> , which stores a table consisting of object address information that is used in lookup by Address.
<i>DW_DEBUG_LINE</i>	This section is called <i>.debug_line</i> , which stores the line number information generated for the compilation units.
<i>DW_DEBUG_LOC</i>	This section is called <i>.debug_loc</i> , which stores the location lists information.
<i>DW_DEBUG_ABBREV</i>	This section is called <i>.debug_abbrev</i> , which stores abbreviation declarations.
<i>DW_DEBUG_MACINFO</i>	This section is called <i>.debug_macinfo</i> , which stores macro information.
<i>DW_DEBUG_REF</i>	This section is called <i>.WATCOM_references</i> , which contains information about the symbols of every instructions in the source files.
<i>DW_DEBUG_MAX</i>	Defined for convenience; it is the number of sections.

7.1 Performance Considerations

The DW library does it's best to try and group CLIWrite operations together into one larger CLIWrite, and to try and avoid using CLISearch. But the library does not go out of it's way to provide this massaging of output. The client should attempt to buffer the data itself. CLISearch is most often called on the DW_DEBUG_INFO, and the DW_DEBUG_LOC sections. The other sections may have one CLISearch performed at the DW_Fini stage, and the seek will be to the zero offset. The client might wish to optimize performance for only the DW_DEBUG_INFO and the DW_DEBUG_LOC sections.

7.1.1 CLISseek

void CLISseek(uint section, long offset, uint mode);

Repositions the pointer in `section` so that subsequent output occurs at the new pointer.

<i>Mode</i>	<i>Description</i>
-------------	--------------------

<i>DW_SEEK_SET</i>	The position is set to the absolute location <code>offset</code> .
--------------------	--

<i>DW_SEEK_CUR</i>	<code>offset</code> is added to the current position.
--------------------	---

<i>DW_SEEK_END</i>	The position is set to <code>offset</code> bytes from the current end of <code>section</code> .
--------------------	---

7.1.2 CLITell

long CLITell(uint section);

Return the offset of the next byte to be written to the section.

7.1.3 CLIReloc

void CLIReloc(uint section, uint reloc_type, ...);

Even when writing BROWSER information, relocations such as `DW_W_LOC_PC` may be asked for. This is because the DWARF format requires the presence of certain fields to indicate something specific about a record. For example, if a subroutine record doesn't have a low pc then it is assumed to be a declaration of the subroutine rather than a definition.

Parameter

<i>section</i>	The section to write a relocation entry to.
----------------	---

<i>reloc_type</i>	The type of the relocation, as follows:
-------------------	---

<i>DW_W_LOW_PC</i>	Emit a <code>dw_targ_addr</code> . Used by various entry points to get the low pc address of an object.
--------------------	---

<i>DW_W_HIGH_PC</i>	Emit a <code>dw_targ_addr</code> . Used by various entry points to get the high pc address of an object.
---------------------	--

<i>DW_W_STATIC</i>	Emit a <code>dw_targ_addr</code> . This relocation has an extra parameter of type <code>dw_sym_handle</code> . This parameter is the target of the relocation; the offset of the symbol should be generated. This is used any time a location expression involving a <code>DWLocStatic</code> is generated.
--------------------	---

<i>DW_W_SEGMENT</i>	Emit a <code>dw_segment</code> . This relocation has an extra parameter of type <code>dw_sym_handle</code> . It indicates that the segment portion of the address of the symbol should be generated. This is used any time a location expression involving a <code>DWLocSegment</code> operation is generated.
---------------------	--

DW_W_LABEL Emit a `dw_targ_addr`. Used by `DWLabel`.

DW_W_SECTION_POS Emit a `uint_32`. This relocation has an extra parameter of type `uint` called `targ_sect`. `targ_sect` parameter is the number of a section for which the current offset is the target of the relocation. The relocation is emitted into `section`.

DW_W_DEFAULT_FUNCTION Emit a `dw_targ_addr`. Used by `DWFormalParameter`.

DW_W_ARANGE_ADDR Emit a `dw_targ_addr`. Used by `DWAddress`.

DW_W_UNIT_SIZE Emit an `uint_32` that is the number of bytes of code in the current compilation unit.

DW_W_MAX Defined for convenience. This enumerated type starts at 0 and goes to `DW_W_MAX`.

7.1.4 CLIWrite

```
void CLIWrite( uint section, const void *block, size_t len );
```

Writes out the debugging information.

Parameter

section The section to which the debugging information is written.

block Points to the debugging information block.

len Length of the debugging information block.

7.1.5 CLIAlloc

```
void *CLIAlloc( size_t size );
```

Allocates a memory block of size `size` for the library and returns its address. This function cannot return `NULL`.

7.1.6 CLIFree

```
void CLIFree( void *blk );
```

Free the block pointed by `blk`.

8 Examples

This section needs a major rewrite.

The example below shows what functions should be called in order to store the debugging information for this C program.

N.B. In this example, for all the `CLIWrite()` calls, only the section id is accurate. Also for all `DWLineNum()` calls, the advances in machine instruction address are inaccurate.

test.c:

```
1 #include <stdlib.h>
2 int    a;
3 typedef near char NCHAR;
4 void main()
5 {
6     NCHAR    b;
7     b := 5;
8 }
```

Functions called by the client and the DWARF library.

Client:

```
cli_id = DWInit( DW_LANG_C89, DW_CM_DEBUGGER, "test.c",
                "c:\mydir", 0x123, 1, CLIILoc, CLIType,
                CLIName, CLIWrite, CLIAlloc, CLIFree );
```

DWARF Library:

```
/* Initialize the .debug_line section */
CLIWrite( DW_DEBUG_LINE, 0, &info, 20, block );

/* Initialize the .debug_abbrevs section */
CLIWrite( DW_DEBUG_ABBREVS, 0, &info, 50, block );

/* Initialize the .debug_pubnames section */
CLIWrite( DW_DEBUG_PUBNAMES, 0, &info, 50, block );

/* Initialize the .debug_aranges section */
CLIWrite( DW_DEBUG_ARANGES, 0, &info, 50, block );

/* Write all strings to the string table */
CLIWrite( DW_DEBUG_STR, 0, &info, 17, block );
```

Client:

```
#include <stdlib.h>
```

```
DWLineNum( cli_id, DW_LN_STMT|DW_LN_BLK, 1, 1, 0 );
DWIncl( id, "stdlib.h" );
...Function calls for "stdlib.h"...
DWInclFini( cli_id );
```

DWARF Library:

```
CLIWrite( DW_DEBUG_LINE, 0, &info, 28, block );
CLIWrite( DW_DEBUG_INFO, 30, &info, 12, block );
```

Client:

int a;

```
DWLineNum( cli_id, DW_LN_STMT, 1, 1, 4 );
a_dw_handle = DWModSym( cli_id, a_cg_handle, DW_SM_VAR,
                        DW_SM_GLO|DW_SM_FILE, DW_SM_NULL );
```

DWARF Library:

```
name = CLIName( a_cg_handle );
/* It returns the string "a". */
type = CLIType( a_cg_handle );
/* It returns DW_FT_INTEGER. */
loc = CLILoc( a_cg_handle );
CLIWrite( DW_DEBUG_LINE, 0, &info, 28, block );
CLIWrite( DW_DEBUG_INFO, 0, &info, 24, block );
CLIWrite( DW_DEBUG_PUBNAMES, 0, &info, 12, block );
```

Inside CLILoc():

```
loc_id = DWLocInt();
DWLocAtom( cli_id, a_cg_handle, DW_LOC_STATIC );
/* The actual address will be filled in by the client when
   the debugging information is written to the object file.*/
a_loc_hd = DWLocFini( loc_id );
return a_loc_hd;
```

Client:

typedef near char NCHAR;

```
DWLineNum( cli_id, DW_LN_STMT, 1, 1, 14 );
mod_handle = DWMod( cli_id, DW_FT_CHAR, DW_MOD_NEAR );
nchar_handle = DWModSym( cli_id, nchar_cg_handle,
                         DW_SM_TYPEDEF, DW_SM_NULL, DW_SM_NULL );
```

DWARF Library:

```
name = CLIName( nchar_cg_handle );
/* It returns the string "NCHAR". */
type = CLIType( nchar_cg_handle );
/* It returns mod_handle. */
CLIWrite( DW_DEBUG_LINE, 0, &info, 20, block );
CLIWrite( DW_DEBUG_INFO, 0, &info, 24, block );
```

Client:

void main()

```
DWLineNum( cli_id, DW_LN_DEFAULT, 1, 1, 23 );
pro_handle = DWBegProc( cli_id, DW_SB_NEAR_CALL, DW_FT_VOID,
                        ret_loc_hd, DW_LOC_NULL,
                        DW_SB_GLOBAL_SUB|DW_SB_FUNC_PROTOTYPE );
```

In order to get `ret_loc_ad`:

```
loc_id = DWLocInit();
DWLocAtom( cli_id, some_cg_handle, DW_LOC_STATIC );
/* Assume that the return address of main() is stored
   in a symbol with some_cg_handle as its handle.
   The actual address will be filled in by the
   client when the debugging information is written
   to the object file. */
ret_loc_ad = DWLocFini( cli_id );
```

DWARF Library:

```
CLIWrite( DW_DEBUG_LINE, 0, &info, 20, block );
```

Client:

```
{
    DWLineNum( cli_id, DW_LN_BLK, 1, 1, 0 );
```

DWARF Library:

```
CLIWrite( DW_DEBUG_LINE, 0, &info, 24, block );
```

Client:

NCHAR b;

```
DWLineNum( cli_id, DW_LN_STMT, 1, 1, 10 );
b_handle = DWModSym( cli_id, b_cg_handle, DW_SM_VAR,
                    DW_SM_NULL, DW_SM_LOC|DW_SM_ROUT );
```

DWARF Library:

```
loc = CLILoc( b_cg_handle );
name = CLIName( b_cg_handle );
/* It returns the string "b". */
type = CLIType( b_cg_handle );
/* It returns nchar_handle. */
CLIWrite( DW_DEBUG_LINE, 0, &info, 20, block );
```

Inside `CLILoc()`:

```
loc_id = DWLocInt();
DWLocAtom( cli_id, b_cg_handle, DW_LOC_STACK );
/* The offset from stack frame base will be filled in by
   the client when the debugging information is written
   to the object file. */
b_loc_hd = DWLocFini( loc_id );
return b_loc_hd;
```

Client:

b := 5;

```
DWLineNum( cli_id, DW_LN_STMT, 1, 4, 14 );
```

DWARF Library:

```
CLIWrite( DW_DEBUG_LINE, 0, &info, 24, block );
```

Client:

```
}
```

```
DWLineNum( cli_id, DW_LN_DEFAULT, 1, 1, 4 );
DWEndProc( cli_id, pro_handle );
main_handle = DWModSym( cli_id, main_cg_handle, DW_SM_SUB,
                        DW_SM_NULL, DW_SM_NULL );
```

DWARF Library:

```
name = CLIName( main_cg_handle );
/* It returns the string "main" */
type = CLIType( main_cg_handle );
/* It returns pro_handle */
CLIWrite( DW_DEBUG_LINE, 0, &info, 24, block );
CLIWrite( DW_DEBUG_INFO, -50, &info, 86, block );
CLIWrite( DW_DEBUG_REF, 0, &info, 12, block );
CLIWrite( DW_DEBUG_PUBNAMES, 0, &info, 12, block );
/* For the global object "main" */
```

Client:

```
DWFinis( cli_id );
```

DWARF Library:

```
CLIWrite( DW_DEBUG_LINE, 0, &info, 24, block );
CLIWrite( DW_DEBUG_INFO, -120, &info, 54, block );
```

9 Revision History

- | | |
|-----------------------|--|
| <i>Draft 5</i> | Changed the arguments to a number of the function calls for use with draft 5 of dwarf. |
| <i>Draft 6</i> | Changed the arguments to a number of the function calls for use with draft 6 of dwarf. |

